Goals, phase 1

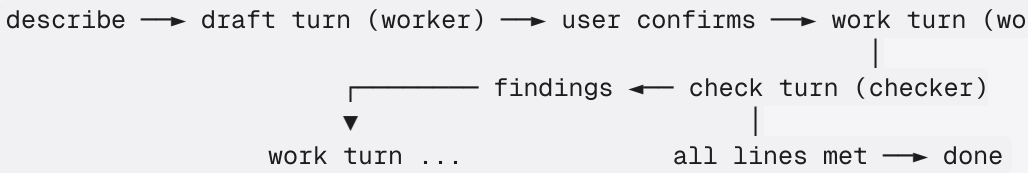
A goal is something the user wants done end to end. They describe it in their own words; the model that will do the work drafts a brief (the goal restated, a checklist of what "done" means, at most one question); the user confirms it; then the work runs in rounds until a second provider agrees every line of the checklist is true.

Mockup: `docs/mockups/missions-phase1.html` (filename predates the Goals name; layout mock for phase 1).

Principles

- **The models decide.** No step counts, time limits or token budgets by default. A round is one ordinary turn, run the way each CLI already runs best, and it ends when the model ends it.
- **Nobody grades their own work.** A round is over when the worker says so; the goal is over only when a different provider has checked it against the agreed checklist.
- **The user is asked, not surprised.** Rocket pauses and asks when the same gap comes back without progress, when a model needs a decision only the user can make, or when a usage window reaches 90% (the one default guard, there to protect the subscription; it can be changed under Advanced).
- **A goal is a chat.** It reuses the transcript, the handoff between providers, browser tabs, teamwork notes and the per-turn commit. Nothing about how a provider works leaks into the goal runner: it only calls the hub.

How a goal runs



1. **Draft.** `goal.draft` creates a chat in the folder and sends the worker a draft prompt: read what you need, then call `goal_report` with the brief, and change nothing. The goal waits in `confirming`. If the turn ends without a brief, the user gets the model's reply and a Retry.
2. **Confirm.** The user edits the goal, the lines and the answer to the question, then presses Start (`goal.start`). Advanced holds the worker, the checker and the usage pause.
3. **Work.** The worker's session already read the folder while drafting, so round 1 resumes it with the agreed brief: work until it is all true, check your own work, then call `goal_report` with `done`, or with `needs_you` and the question.
4. **Check.** When the work turn ends, the runner sends the checker a check prompt in the same chat. The hub's handoff gives it the worker's turns, with the files they read and edited. It is told to verify every line itself (run the commands, open the page, read the code), not to take the worker's account on trust, to change nothing, and to call `goal_report` with a verdict per line, its evidence, and what is still wrong.

5. **Next round.** If every line is met, the goal is `done`. Otherwise the checker's findings become the worker's next message, along with any note the user added in the meantime.

When the runner pauses (status `needs_you`)

- The worker reported `needs_you`: its question is shown, and the answer is the next round's message.
- **No progress:** two checks in a row leave the same lines unmet and the second met no more lines.
- **Usage:** before each turn the runner reads the provider's usage (`hub.usage`). At or past the pause level, the goal waits and resumes on its own at the window's `resetsAt`, if one is known.
- A turn failed. One retry first, then pause with the error.
- The app was closed mid-round. On start, a goal whose turn was interrupted is paused with "Rocket was closed during round N", and Resume picks it up.

A worker turn that ends without calling `goal_report` is treated as "done for now" and goes to the checker; models forget tools, and the checker decides anyway. A check without a verdict is asked once more, then the goal pauses.

Choosing the checker

The user's current provider works. The checker is another provider whose usage reads as signed in and has the most headroom. With no second provider signed in, the worker checks in a fresh session and the goal says so: weaker, but better than no check.

The one new tool: `goal_report`

The worker must report an outcome and the checker must report a verdict, and both must be structured, so this is a tool (CLAUDE.md: add a tool only for something a model must do). It is one tool, not three, because every schema costs every session prompt space. The runner knows which role the calling chat's running turn has, and rejects arguments that do not fit it.

```
goal_report({
  // draft: the brief
  goal?: string
  criteria?: { text: string; how: string }[]   // how: "runs it", "screenshot", "tried i
  question?: { text: string; options: string[] }
  // work: the outcome
  status?: 'done' | 'needs_you'
  summary?: string
  // check: the verdict
  verdicts?: { line: number; met: boolean; evidence: string }[]
  findings?: string
})
```

Outside a goal it answers that there is no goal running in this chat.

Protocol (`src/shared/types.ts`)

An outline; the types in `src/shared/types.ts` are the source of truth (they also hold `next`, the turn a paused goal takes when it resumes, and `resumeAt` for a goal waiting on usage).

```
export type GoalStatus = 'drafting' | 'confirming' | 'working' | 'checking' | 'needs_yo
export type GoalRole = 'draft' | 'work' | 'check'
export type GoalCriterion = { text: string; how: string; met: boolean | null; metInRound
export type GoalRound = {
  n: number
  workTurnId: string
  checkTurnId: string | null
  workSummary: string | null      // from the worker's report, or its last message
  findings: string | null
  verdicts: { line: number; met: boolean; evidence: string }[]
  files: string[]                 // changed in the work turn, from teamwork's snapshot
  startedAt: number
  endedAt: number | null
}
export type Goal = {
  id: string
  chatId: string
  cwd: string
  request: string                 // the user's own words
  title: string
  goal: string
  criteria: GoalCriterion[]
  question: { text: string; options: string[]; answer: string | null } | null
  worker: { provider: ProviderId; model: string | null }
  checker: { provider: ProviderId; model: string | null }
  pauseAtPercent: number          // 90 by default
  status: GoalStatus
  reason: string | null           // why it is paused or needs the user, as a full sent
  steer: string[]                 // notes queued for the next work round
  rounds: GoalRound[]
  createdAt: number
  updatedAt: number
}
```

`Turn` gains `goalRole: GoalRole | null`, so the transcript can label check turns and the renderer can hide the app's prompts behind "Rocket asked Codex to check round 2".

Events: `{ type: 'goal'; goal: Goal }` whenever a goal changes.

Requests: `goals.list`, `goal.draft {cwd, text, provider, model}`, `goal.update {goalId, goal, criteria, answer, worker, checker, pauseAtPercent}` (while confirming), `goal.start`, `goal.pause`, `goal.resume`, `goal.stop`, `goal.steer {goalId, text}`, and deleting the chat deletes the goal.

Store (`src/host/store.ts`)

One `goals` table: the goal as one row of JSON (rounds included), keyed by id with its `chat_id`. A goal is small and is read and written whole. Turns gain a `goal_role` column, added in place to stores made before it. Deleting the chat deletes its goal (`ON DELETE CASCADE`).

Host

- `src/host/goals.ts`: `GoalRunner`, the state machine above. It is given a narrow interface (`send`, `stop`, `usage`, `providers`) rather than the hub itself, so it is tested with a fake and no

CLI, like the translators. It holds the reports `goal_report` hands it until the turn that made them ends.
- `src/host/tools/goal.ts`: the tool, which hands its arguments to the runner.
- `src/host/index.ts`: builds the runner, adds the tool, routes `goal.*` requests, and calls `goals.turnEnded(turn)` from `onEvent` beside teamwork's.
- `src/host/hub.ts`: `send` takes an optional `goalRole`, stored on the turn. Nothing else changes.
- The prompts the runner sends (draft, work, check) live in `goals.ts` as constants, each a few plain sentences, and are tested for what they must contain.

Renderer

- Rail: a Goals button with a count of goals working or needing the user, and a flyout like history.
- `NewGoal.tsx`: the two steps. Describe: one text box, the folder, a few example chips. Confirm: the editable brief, the question as buttons, "How it will run", Advanced.
- **Layout: conversation | artifact.** A goal opens as a chat: the transcript on the left, as for any chat, with each turn labelled (round 2 · work, round 2 · check) and the app's own prompts folded into one line. A check turn ends in a verdict card with "See the evidence". The composer queues a note for the next round instead of sending a message.
- **The chat card splits, like an artifact.** The conversation is on the left and the goal on the right, inside the same card, with a draggable divider. `GoalPane.tsx` shows status, the goal, the checklist with each line's state, rounds that open in place to show the checker's evidence, screenshot and files, and "Left to the models", with Pause/Resume/Stop and a ✕ in its header. ✕ hides the pane (the goal keeps running); the "Goal · round 3" chip in the chat header brings it back. The pane's width is remembered.
- **The browser stays a separate card, toggled as today**, from a button in the chat header too. With it open the split chat takes about 60% of the width; with it closed the split takes it all and the goal pane widens. The page is a native view whose bounds already follow the layout.
- A system notification when a goal finishes or needs the user (the page's `Notification`).

Build order

1. Types and store, with store tests.
2. `GoalRunner` with a fake hub: drafting, rounds, every pause, restart recovery, checker choice.
3. `goal_report`, `goalRole` on turns, wiring in `index.ts`.
4. Renderer: flyout, dialog, goal view, notifications.
5. README (a Goals section) and CLAUDE.md's "Where things are".

Later in phase 1, if time allows: **Undo to here**, restoring the folder to a round's end from the snapshot teamwork already takes, through a new `restore` in `git.ts`, after a confirm.

Left for later phases

- `check:ui` **for goals.** `UiTestAdapter` should learn to call `goal_report` from a script, so a goal can run draft → work → check (one line failing) → work → check → done with no real model.

- Approval gates on irreversible actions, scheduling and triggers, curated memory and skills, notifications to the phone.